

Atlas AQAR

أطلس عقار

Agentic AI-Driven Real-Estate Research & Investment Intelligence System

Project Documentation — Plan · Stakeholder Analysis · WBS · PDF Structure

DOCUMENT VERSION v1.0 (Draft Baseline)	**PROGRAM** DEPI Round 5	**TIMELINE** 4 Weeks
REPOSITORY nhahub/NHA-5-072	**DOMAIN TRACK** AI / Machine Learning	**STATUS** Under Active Development

LangGraph Orchestration · Multi-Agent Cognitive Loop · Deterministic Financial Engine · ChromaDB Vector Store · FastAPI Service · Streamlit UI · Docker Compose

Digital Egypt Pioneers Initiative (DEPI) — Project NHA-5-072 · Confidential Internal Document

Table of Contents

Overview of the project documentation sections and reference layout.

1 Executive Summary Section 1

2 Project Overview Section 2

2.1 Background & Problem Statement §2.1

2.2 Vision, Mission & Objectives §2.2

2.3 Project Scope & Boundary Exclusions §2.3

2.4 Technology Stack & Architecture Layers §2.4

2.5 Performance Targets & Baseline Metrics §2.5

3 Stakeholder Analysis Section 3

3.1 Internal Stakeholders Analysis §3.1

3.2 External Stakeholders Analysis §3.2

3.3 Power / Interest Classification Matrix §3.3

3.4 Comprehensive Communication Plan §3.4

3.5 Strategic Stakeholder Engagement Guidelines §3.5

4 Project Plan Section 4

4.1 Chronological Project Milestones §4.1

4.2 Key Deliverables Register §4.2

4.3 Risk Register & Proactive Mitigations §4.3

4.4 Assumptions & Operational Constraints §4.4

5 Work Breakdown Structure (WBS) Section 5

5.1 Week 1: Foundations, Modeling & Infrastructure §5.1

5.2 Week 2: Core Agent Intelligence & Analytical Engine §5.2

5.3 Week 3: Integration, API Services & User Interface §5.3

5.4 Week 4: Benchmarking, Containerization & Release §5.4

5.5 Summary Matrix of Functional Workstreams §5.5

6 Project's PDF Document Structure & Standards Section 6

SECTION 1

Executive Summary

Strategic overview, project mission, operational value, and measured performance baselines.

🏛️

SYSTEM CATEGORY

Agentic AI Platform

Stateful multi-agent system executing real-estate investment research via LangGraph.

⏱️

DELIVERY SPRINT

4 Sprints / 4 Weeks

Rigorous 28-day lifecycle from schema foundation to dockerized deployment.

🎯

AFFILIATION

DEPI Round 5

Digital Egypt Pioneers Initiative — Track NHA-5-072 under AI & Machine Learning.

System Concept & Core Value Proposition

Atlas AQAR (أطلس عقار) is a production-grade, bounded multi-agent AI system designed to conduct autonomous real-estate research, due-diligence, and investment analysis tailored specifically to the Egyptian property market. Moving beyond standard single-turn LLM chatbots, Atlas AQAR implements a stateful, cyclic **LangGraph** graph workflow that breaks complex investment prompts into structured sub-tasks, gathers grounded evidence via vector retrieval and external verification, performs exact financial calculations, and passes all outputs through an automated editorial Critic agent.

A foundational design principle of Atlas AQAR is *deterministic financial computation*: all mortgage amortization, rental yields, capital growth projections, and sensitivity models are executed exclusively in dedicated mathematical algorithms, entirely preventing LLM arithmetic hallucinations. Every claim made in generated investment reports is directly tagged with an audit-ready Evidence ID (e.g., `[E001]`).

Empirical Performance Baseline

TASK COMPLETION RATE

100.0%

5 of 5 benchmark scenarios successfully fulfilled against target ≥80%.

CITATION PRECISION

100.0%

Zero uncited factual statements detected across verified evaluation runs.

FINANCIAL DETERMINISM

100.0%

Zero financial hallucination; mathematical outputs strictly match pure Python calculations.

CRITIC EVALUATION SCORE

100.0%

100% compliance across 7-point quality, evidence, and completeness criteria.

UNSUPPORTED CLAIM RATE

0.0%

Completely eliminated ungrounded assertions through evidence arsenal gating.

OFFLINE BENCHMARK LATENCY

0.12 s

High-speed evaluation pipeline executing 5 full research workflows.

Documentation Scope & Continuous Iteration: This document (v1.0 Baseline) defines the foundational Project Plan, Stakeholder Matrix, 4-Week Work Breakdown Structure (WBS), and target PDF Publication Architecture. In alignment with modern engineering practices, subsequent releases (v1.1, v1.2, and v2.0 Final) will iteratively document development outcomes, updated benchmark metrics, and production releases.

SECTION 2

Project Overview

Market challenges, objectives, system boundaries, technology selections, and architectural foundations.

2.1 Background & Problem Statement

The Egyptian real-estate ecosystem is among the most dynamic investment landscapes in North Africa and the Middle East. However, individual and institutional investors face severe informational friction:

- **Data Fragmentation:** Unit listings, pricing indices, and developer track records are scattered across disparate websites, social groups, and informal brokers.
- **Information Asymmetry:** Potential buyers lack verified data on developer delivery track records, historical delays, and construction solvency.
- **Financial Calculation Opacity:** Real-estate installment plans often include hidden maintenance deposits, backloaded balloon payments, and fluctuating interest formulas that retail investors struggle to assess.
- **GenAI Unreliability:** Standard LLMs hallucinate property prices, invent developers, and introduce gross calculation errors when asked for real-estate ROI forecasts.

The Solution: Atlas AQAR provides an automated, evidence-grounded research co-pilot that combines semantic vector search, deterministic financial amortization, multi-agent cognitive critique, and professional report generation into a unified platform.

2.2 Vision, Mission & SMART Objectives

Vision: To become the leading trusted, verifiable AI intelligence engine for property research and real-estate investment decision-making in Egypt.

Mission: To democratize institutional-grade property due diligence by combining agentic reasoning with rigorous mathematical determinism and empirical evidence verification.

OBJECTIVE 1 (O1)

Autonomous Task Decomposition

Deconstruct user investment queries into 3–6 dependency-ordered sub-tasks executed systematically.

OBJECTIVE 2 (O2)

Deterministic Financial Engine

Deliver 100% accurate mortgage amortization, ROI, and sensitivity analysis via non-LLM Python algorithms.

OBJECTIVE 3 (O3)

Evidence-Grounded Reporting

Enforce mandatory citation of Evidence IDs (`[E001]`) for every quantitative claim and factual finding.

OBJECTIVE 4 (O4)

Automated Self-Critique & Refinement

Incorporate a dedicated Critic agent applying a 7-point evaluation rubric to guarantee report quality.

OBJECTIVE 5 (O5)

Multi-Model LLM Agnosticism

Support dynamic switching between OpenAI, Anthropic, Google Gemini, and fully offline mock LLMs.

OBJECTIVE 6 (O6)

Modular Production Architecture

Deploy via FastAPI backend, Streamlit interactive dashboard, and Docker Compose orchestration in 4 weeks.

2.3 Project Scope & Boundary Exclusions

✓ FULLY IN SCOPE (V1.0)

- Multi-agent LangGraph workflow (Planner, Researcher, Analyzer, Writer, Critic).
- Curated vector store (ChromaDB) indexing property listings, developer profiles, and market reports.
- Deterministic financial engine (Installments, Yields, ROI, Scenarios).
- FastAPI REST service with versioned API endpoints (`/api/v1/research`, `/api/v1/financial`, etc.).
- Streamlit interactive web dashboard with 6 specialized analytical views.
- Automated evaluation suite with benchmark scenarios.
- Multi-container Docker Compose deployment.

✗ BOUNDARY EXCLUSIONS (V1.0)

- Real-time automated web scraping of dynamic listing portals without API licenses.
- User authentication, multi-tenant RBAC, and subscription billing gateways.
- Mobile native applications (iOS / Android).
- Direct integration with external banking / mortgage approval APIs.
- Multi-language report localization (Arabic report synthesis planned for v2.0).
- Automated legal contract drafting or conveyancing advice.

2.4 Technology Stack & Architecture Layers

ARCHITECTURAL TIER	PRIMARY TECHNOLOGY	VERSION	ROLE & ARCHITECTURAL RESPONSIBILITY
Agentic Orchestration	LangGraph & LangChain-Core	1.2.14 / 1.6.9	Stateful cyclic graph orchestration, agent execution, conditional edge routing.
Data Contracts & Validation	Pydantic v2	2.14.0	Strict typed data validation for GraphState, EvidenceArsenal, and analytical payloads.
Vector Retrieval & Embeddings	ChromaDB & HuggingFace	1.5.9 / 2.2.0	Persistent semantic document indexing, dense vector search, similarity ranking.
Financial Analytics	Pure Python Math Engine	3.11+	Deterministic mortgage amortization, net yields, capital appreciation, sensitivity grid.

ARCHITECTURAL TIER	PRIMARY TECHNOLOGY	VERSION	ROLE & ARCHITECTURAL RESPONSIBILITY
Backend REST API	FastAPI & Uvicorn	0.143.0 / 0.54.0	Async REST service exposing workflow execution, health diagnostics, and OpenAPI specs.
Interactive Dashboard	Streamlit & Plotly	1.65.0 / 7.1.0	Analytical user interface, report rendering, interactive financial modeling graphs.
Testing & QA	pytest & pytest-asyncio	9.1.1 / 1.4.0	Automated unit testing, integration workflows, and benchmark assertions.
Container Infrastructure	Docker & Docker Compose	Latest	Multi-service containerization, environment reproducibility, volume persistence.

2.5 Performance Targets & Baseline Metrics

EVALUATION DIMENSION	EVALUATION METRIC	TARGET THRESHOLD	MEASURED BASELINE	AUDIT RESULT
Task Completion	Sub-task execution rate	≥ 80.0%	100.0%	PASSED
Fact Verification	Citation Precision Score	> 90.0%	100.0%	PASSED
Financial Reliability	Zero-hallucination accuracy	100.0%	100.0%	PASSED
Editorial Quality	Critic 7-point pass rate	≥ 85.0%	100.0%	PASSED
Hallucination Gating	Unsupported claim frequency	≤ 2.0%	0.0%	PASSED
Evaluation Speed	Offline benchmark execution	< 5.0 s	0.12 s (5 cases)	PASSED

SECTION 3

Stakeholder Analysis

Identification, impact evaluation, power-interest classification, and communication management plan.

3.1 Internal Stakeholders Analysis

STAKEHOLDER GROUP	ORGANIZATIONAL ROLE	CORE PROJECT INTEREST	POWER	INTEREST	SUCCESS CRITERIA & EXPECTATIONS
Engineering Team	System Implementation	Clean modular architecture, modern AI tech stack	High	High	Clear specs, decoupled workstreams, robust CI/CD, unblocked sprints.
Project Lead / Scrum Master	Delivery & Alignment	On-time milestone delivery, sprint velocity	High	High	Transparency in progress, early risk escalation, milestone governance.
AI / ML Architect	Technical Direction	System resilience, determinism, agentic bounds	High	High	Strict adherence to Pydantic contracts and LangGraph cyclic patterns.
QA & Verification Lead	Quality Assurance	Defect elimination, benchmark repeatability	Medium	High	Comprehensive test coverage across edge cases, reproducible benchmarks.
DevOps & Infra Lead	Deployment Operations	Container stability, zero configuration drift	Medium	Medium	Hermetic Docker Compose builds, volume persistence, isolated environments.
Technical Documentation Lead	Knowledge Management	Accurate publications, professional reports	Low	Medium	Systematic documentation versioning (v1.0 to v2.0), precise API specs.

3.2 External Stakeholders Analysis

STAKEHOLDER ENTITY	CATEGORY	INSTITUTIONAL INTEREST	POWER	INTEREST	PRIMARY ENGAGEMENT OBJECTIVE
DEPI Leadership & Mentors	Governance / Supervisory	High-caliber national AI graduation showcase	High	High	Deliver compliant, demonstrable AI product meeting all program standards.
National Housing Authority (NHA)	Institutional Beneficiary	Data-driven real-estate analytics tool	High	High	Provide verifiable, audit-proof housing due-diligence capabilities.
Real-Estate Investors (End Users)	Target Consumer Base	Actionable, grounded, trustworthy reports	Medium	High	Eliminate investment uncertainty with transparent financial schedules.
Property Developers	Market Ecosystem	Accurate portrayal of project timelines and specs	Medium	Medium	Maintain factual data accuracy regarding pricing and payment terms.
Academic & Peer Reviewers	Advisory / Academic	Technical novelty, code rigor, repeatability	Low	Medium	Publish clear architectural documentation and benchmark proofs.
LLM Infrastructure Providers	External Technology Vendor	API consumption compliance and quota usage	Medium	Low	Maintain bounded token budgets (100k cap) and offline fallback modes.

3.3 Power / Interest Classification Matrix

MONITOR (LOW INTEREST, HIGH POWER)

LLM Infrastructure Providers, Regulatory Bodies

Ensure API stability and compliance; track usage quotas and maintain offline fallback modes.

MANAGE CLOSELY (HIGH INTEREST, HIGH POWER)

DEPI Leadership, NHA Stakeholders, Project Lead, AI Architect

Primary project drivers. Require weekly milestone updates, prototype demos, and formal milestone reviews.

KEEP INFORMED (LOW INTEREST, LOW POWER)

Academic Mentors, Technical Documentation Team

Keep apprised of project versions, architectural ADRs, and documentation snapshots.

KEEP SATISFIED (HIGH INTEREST, MEDIUM/LOW POWER)

Real-Estate Investors, Engineering Team, QA & DevOps Leads

Ensure user-friendly UI, deterministic accuracy, sprint satisfaction, and unblocked tasks.

3.4 Comprehensive Communication Plan

AUDIENCE	COMMUNICATION CHANNEL	FREQUENCY	ARTIFACT / CONTENT	RESPONSIBLE OWNER
Engineering Team	GitHub Issues & Stand-up	Daily	Task burndown, blocker resolution, PR reviews	Project Lead
DEPI Program Leadership	Formal Progress Briefing	Weekly (Fridays)	Milestone status, risk register updates, deliverables	Project Lead / AI Architect
NHA Institutional Partners	Live Product Demonstration	Bi-weekly	Working system demo, feedback logs, compliance check	Project Lead
Peer Reviewers & Mentors	Documentation Drive & GitHub	Per Milestone	Architecture diagrams, benchmark tables, code reviews	Technical Documentation Lead
End Users / Investors	Interactive Dashboard Tour	Release Milestone	User Guide, quick-start videos, sample reports	QA / Product Lead

3.5 Strategic Stakeholder Engagement Guidelines

GOVERNANCE ENGAGEMENT PROTOCOL

All formal project submissions to DEPI leadership are accompanied by a version-stamped PDF document, reproducible test scripts, and an unlisted demonstration recording. Any architectural deviations from the original proposal require prior review.

TECHNICAL COMMUNITY PROTOCOL

Open communication via structured GitHub PR templates. Every PR must pass automated unit tests (`pytest tests/unit/`) and linting before merge approval. Weekly retrospective meetings capture lessons learned.

SECTION 4

Project Plan

Milestone roadmap, deliverables catalog, comprehensive risk management, and operational boundaries.

4.1 Chronological Project Milestones

WEEK 1 · DAYS 1–7

Milestone 1 (M1) — Environment & Data Foundation Baseline

Repository established at `nhahub/NHA-5-072`. Pydantic v2 domain schemas finalized. Raw property and developer datasets curated and indexed in ChromaDB vector database. System configuration layer and benchmark harness operational.

WEEK 2 · DAYS 8–14

Milestone 2 (M2) — Agent Suite & Deterministic Engine Core

Implementation of 5 LangGraph agent nodes (Planner, Researcher, Analyzer, Writer, Critic). Full 8-tool registry active. Deterministic financial amortization engine completed and validated with 100% test coverage. Cognitive cyclic loop verified.

WEEK 3 · DAYS 15–21

Milestone 3 (M3) — API Services & Interactive Web Dashboard

FastAPI REST service operational with `/api/v1/` routes. Streamlit frontend completed featuring 6 analytical panels (Research Console, Verified Report Viewer, Property Grid, Financial Calculator, Evidence Inspector, and Observability Monitor).

WEEK 4 · DAYS 22–28

Milestone 4 (M4) — Containerization, Benchmarks & Final Release

Multi-service Docker Compose deployment validated. Formal evaluation benchmarks executed and performance metrics cataloged. Technical documentation package (v1.0) published. Final Git tag pushed and submitted.

4.2 Key Deliverables Register

ID	DELIVERABLE NAME	CATEGORY	TARGET WEEK	LIFECYCLE STATUS
D01	Official GitHub Repository (nhahub/NHA-5-072)	Source Code	Week 1	Completed
D02	Pydantic v2 Core Data Models & GraphState	Architecture	Week 1	Completed
D03	ChromaDB Vector Store & Ingestion Pipeline	Data Infra	Week 1	Completed
D04	5-Agent LangGraph Cognitive Workflow	AI Intelligence	Week 2	In Progress

ID	DELIVERABLE NAME	CATEGORY	TARGET WEEK	LIFECYCLE STATUS
D05	8-Tool Specialized Domain Suite	Tools / Engine	Week 2	In Progress
D06	Deterministic Financial Calculation Engine	Analytics Engine	Week 2	In Progress
D07	FastAPI REST API Service (v1 Endpoints)	Backend Service	Week 3	Planned
D08	Streamlit Multi-Panel Interactive Dashboard	User Interface	Week 3	Planned
D09	Multi-Container Docker Compose Deployment	DevOps	Week 4	Planned
D10	Empirical Evaluation & Benchmark Report	Quality / QA	Week 4	Planned
D11	Comprehensive Project Documentation (PDF v1.0)	Documentation	Week 4	Under Review
D12	Final Project Submission & Presentation Deck	Release	Week 4	Planned

4.3 Risk Register & Proactive Mitigations

ID	RISK DESCRIPTION	LIKELIHOOD	IMPACT	SEVERITY	PROACTIVE MITIGATION STRATEGY
R01	External LLM API quota exhaustion / rate limiting	Medium	High	High	Implement automatic offline fallback mock provider; enforce 100k session token budget; support multiple provider API keys.
R02	Upstream dependency breaking changes (LangGraph / ChromaDB)	Low	High	High	Pin exact library versions in `requirements.txt`; isolate dependencies in hermetic virtual environments.
R03	Vector database persistence corruption or re-indexing lag	Low	Medium	Medium	Write idempotent ingestion scripts; persist ChromaDB to dedicated mounted volumes; add health check probe on startup.

ID	RISK DESCRIPTION	LIKELIHOOD	IMPACT	SEVERITY	PROACTIVE MITIGATION STRATEGY
R04	Edge-case financial calculation discrepancies (balloon payments)	Medium	High	Medium	Implement 100% unit test coverage for mathematical engine; enforce strict Pydantic numeric validation; isolate math from LLM prompts.
R05	Inter-workstream integration delays during Week 3	Medium	Medium	Medium	Define typed Pydantic interfaces in Week 1; use mock responses for parallel backend and frontend development.
R06	OS environment discrepancies (Windows PowerShell vs Linux Docker)	Medium	Low	Low	Test Docker Compose builds locally across platforms; provide explicit `.env.example` with platform-independent path resolution.
R07	Streamlit state loss or excessive re-rendering latency	Low	Low	Low	Use `st.session_state` caching for immutable data; paginate property tables; lazy-load complex report graphs.

4.4 Assumptions & Operational Constraints

CORE PROJECT ASSUMPTIONS

- All developers operate on Python 3.11+ with local Docker Desktop installed.
- Curated JSON datasets accurately reflect representative Egyptian real-estate market segments in New Cairo, New Capital, and Sheikh Zayed.
- At least one live API provider key (OpenAI / Gemini / Anthropic) is accessible for validation, while offline mock mode is available for local testing.
- Four-week timeline is firm with fixed weekly sprint reviews.

OPERATIONAL CONSTRAINTS

- Maximum of 10 LangGraph workflow iterations per research request to prevent infinite loops.
- Maximum of 3 Critic revision retries before forced report synthesis.
- Zero arithmetic calculations delegated to LLM text generation.
- System must be fully runnable locally without mandatory paid cloud dependencies.

SECTION 5

Work Breakdown Structure (WBS)

A comprehensive four-week Work Breakdown Structure organized into six specialized functional workstreams. Each workstream represents a distinct, dedicated domain of technical responsibility operating continuously across all four weeks, ensuring clear separation of concerns and end-to-end task accountability.

Workstream Organization: Work is partitioned into six distinct technical domains: *(1) Core Architecture & State Modeling*, *(2) Data Engineering & Vector Store*, *(3) Agentic Intelligence & Cognitive Loops*, *(4) Deterministic Financial Engine*, *(5) Backend API & Services*, and *(6) User Interface, Testing & DevOps*. Dependencies between tracks are handled via strict Pydantic schemas.

Week 1 — Sprint 1: Architecture, Data Engineering & Core Modeling — Days 1 to 7

Workstream 1 ARCHITECTURE & STATE	▸ Architect the master multi-agent LangGraph workflow diagram and cyc ▸ Author `GraphState` , `EvidenceArsenal` , `EvidenceRecord` , `Proper` and validate all 10 core Pydantic v2 data models (▸ Establish runtime guardrail configurations in `config/settings.py` (10
Workstream 2 DATA & KNOWLEDGE BASE	▸ Curate and normalize `data/raw/properties.json` covering New Cairo, New Capital, and Sheikh Zayed developments. ▸ Compile `data/raw/developers.json` with developer solvency histories, delivery timeliness scores, and completed projects. ▸ Build the automated ChromaDB embedding and ingestion pipeline in `scripts/ingest_data.py` with deduplication.
Workstream 3 AGENTIC REASONING	▸ Formulate system prompt templates in `config/prompts/planner.yaml` for deterministic query decomposition. ▸ Draft Planner agent logic converting arbitrary natural-language investor queries into 3–6 dependency-ordered sub-tasks. ▸ Define Pydantic schema validation for intermediate sub-task state updates.

Workstream 4 FINANCIAL ALGORITHMS	▸ Develop mathematical formula specifications for equal monthly amortization, delivery balloon payments, and maintenance funds. ▸ Implement core mortgage calculation algorithms in `src/engine/installments.py` with zero float precision drift. ▸ Write unit test assertions verifying that total installment schedules sum exactly to 100% of purchase price.
Workstream 5 BACKEND API LAYER	▸ Scaffold the FastAPI service structure in `src/api/main.py` with CORS middleware and lifespan events. ▸ Define request and response DTO schemas in `src/api/schemas.py` mapping directly to Pydantic domain models. ▸ Implement the system health check endpoint `GET /api/v1/health` reporting component statuses.
Workstream 6 UI, QA & DEVOPS	▸ Initialize git repository at `nhahub/NHA-5-072`, configure branch protections on `main`, and enforce `.gitignore`. ▸ Configure `pyproject.toml` and `requirements.txt` with pinned dependency versions; setup virtual environment. ▸ Author baseline Dockerfile and docker-compose.yml service skeletons for local development.

Week 2 Sprint 2: Multi-Agent Logic, Tool Registry & Financial Engine

Days 8 to 14

Workstream 1 ARCHITECTURE & STATE	▸ Compile the LangGraph `StateGraph` in `src/graph/workflow.py` connecting all five agents with cyclic loops. ▸ Implement conditional edge routing in `src/graph/edges.py` based on Critic verdicts (`accept` vs `reject`). ▸ Enforce graph-level iteration safeguards and timeout handlers preventing execution deadlocks.
Workstream 2 DATA & KNOWLEDGE BASE	▸ Implement `RAGRetrieverTool` in `src/tools/rag_search.py` querying ChromaDB with top-similarity ranking.

	▸ Implement `PropertySearchTool` and `PropertyDetailsTool` supportir compour filtering (location price, bedroom ▸ Implement `DeveloperLookupTool` extracting historical project completion records and solvency metrics.
Workstream 3 AGENTIC REASONING	▸ Build Researcher agent ReAct execution loop (Thought → Action → Observation) with maximum 8 reasoning steps. ▸ Build Writer agent synthesizing 9-section `[E001]` evidence structured reports with mandatory inline citations. ▸ Build Critic agent applying the 7-point verification rubric, generating actionable revision directives.
Workstream 4 FINANCIAL ALGORITHMS	▸ Develop capital appreciation and net rental yield algorithms `src/eng` in ▸ Build sensitivity `src/engine/scenarios.py` generating conserv scenario analyzer in and aggressive moc ▸ Package `InstallmentCalcTool` , `ROIEstimatorTool` , `ScenarioAn` tools: and
Workstream 5 BACKEND API LAYER	▸ Implement `POST /api/v1/research` endpoint to asynchronously invoke the LangGraph engine. ▸ Implement `GET /api/v1/properties` endpoint supporting paginated multi-parameter discovery queries. ▸ Implement `POST /api/v1/financial` endpoint executing standalone mathematical calculations without LLM overhead.
Workstream 6 UI, QA & DEVOPS	▸ Write `tests/unit/test_financial_engine.py` testing unit test suites in boundary conditions and edge cases. ▸ Write unit `tests/unit/test_tools.py` verifying schema test suites in validity across all 8 tools. ▸ Scaffold Streamlit `src/ui/dashboard.py` with custom CSS application layout in tokens and brand palette.

Week 3 Sprint 3: Full-Stack Integration, Web UI & End-to-End QA

Days 15 to 21

Workstream 1 ARCHITECTURE & STATE	▸ Optimize multi- `src/agents/researcher.py` supporting OpenAI, model provider Gemini, and offline

	adapter in mocks. ▸ Integrate OpenTelemetry instrumentation hooks tracking execution latency, tokens consumed, and graph iterations. ▸ Validate state immutability across cyclic transitions to prevent memory leakage during long runs.
Workstream 2 DATA & KNOWLEDGE BASE	▸ Ingest supplemental market intelligence reports into `data/raw/market_reports.json`. ▸ Implement `WebSearchTool` fallback with data-sanitization guardrails preventing raw HTML injection. ▸ Benchmark ChromaDB retrieval latency and fine-tune embedding chunk sizes for maximum citation precision.
Workstream 3 AGENTIC REASONING	▸ Tune Writer agent prompt engineering to guarantee consistent 9-section report generation under varied inputs. ▸ Refine Critic agent feedback formatting to provide deterministic line-item critique prompts for the Writer. ▸ Validate end-to-end evidence chain integrity: trace every Evidence ID from user output back to raw source.
Workstream 4 FINANCIAL ALGORITHMS	▸ Expand scenario sensitivity engine to support custom inflation rate and rental demand fluctuation multipliers. ▸ Implement currency formatting helpers (EGP / USD) and localized numeric parsers for UI presentation. ▸ Validate financial output serialization against Pydantic OpenAPI schemas.
Workstream 5 BACKEND API LAYER	▸ Finalize error handling middlewares returning standardized JSON error envelopes with trace IDs. ▸ Generate and audit comprehensive OpenAPI (Swagger) interactive documentation at `/docs`. ▸ Implement request rate limiting and query timeout guards for high-concurrency protection.
Workstream 6 UI, QA & DEVOPS	▸ Complete all 6 Streamlit UI panels: Research Console, Report Viewer, Property Grid, Calculator, Evidence Inspector, and Observability Monitor. ▸ Implement interactive Plotly charts: price-per-sqm distributions, installment timeline bars, and ROI gauge. ▸ Author comprehensive integration tests in `tests/integration/test_workflow.py` asserting 100% pipeline success.

Week 4	Sprint 4: Benchmarking, Containerization, Documentation & Submission	Days 22 to 28
Workstream 1 ARCHITECTURE & STATE	▸ Conduct architectural code audit verifying clean decoupling across agents, tools, engine, and API. ▸ Perform memory profiling and ensure zero state leakage over repeated batch runs. ▸ Document final Architecture Decision Records (ADRs) for multi-agent graph orchestration.	
Workstream 2 DATA & KNOWLEDGE BASE	▸ Lock and archive production data snapshots in `data/raw/` with immutable SHA-256 checksums. ▸ Verify ChromaDB volume mounting persistence across Docker container rebuilds. ▸ Author dataset provenance documentation outlining source boundaries and indexing methodology.	
Workstream 3 AGENTIC REASONING	▸ Execute formal evaluation script `scripts/run_evaluation.py` across all benchmark test scenarios. ▸ Tabulate empirical metrics: Task Completion (100%), Citation Precision (100%), Critic Pass Rate (100%). ▸ Capture sample end-to-end report artifacts for technical appendix and showcase portfolios.	
Workstream 4 FINANCIAL ALGORITHMS	▸ Perform mathematical boundary testing across extreme parameter ranges (0% down payments, 15-year terms). ▸ Verify 100% zero-hallucination compliance by cross-comparing engine outputs with external spreadsheet models. ▸ Publish financial calculation methodology guide with formal amortization formulas.	
Workstream 5 BACKEND API LAYER	▸ Execute API stress test with 10 concurrent requests; verify sub-second health check response under load. ▸ Lock FastAPI production configuration with Uvicorn worker settings. ▸ Publish complete API Reference documentation with cURL examples and schema payloads.	
Workstream 6 UI, QA & DEVOPS	▸ Finalize multi-stage Dockerfile and validate `docker-compose up --build` launches API and UI cleanly. ▸ Generate and format Project Documentation PDF (v1.0 Baseline) conforming to PDF standards. ▸ Update `README.md` with Quick Start instructions, architecture diagrams, and push final release tag to GitHub.	

5.5 Summary Matrix of Functional Workstreams

WORKSTREAM DOMAIN	PRIMARY OWNERSHIP AREA	WEEKS ACTIVE	KEY ARTIFACTS DELIVERED
Workstream 1	Architecture, State & Graph Governance	Weeks 1 to 4	LangGraph StateGraph, Pydantic data models, conditional edge routing, ADRs.
Workstream 2	Data Engineering & Knowledge Retrieval	Weeks 1 to 4	Curated real-estate datasets, ChromaDB vector store, RAG search tools.
Workstream 3	Agentic Intelligence & Cognitive Critique	Weeks 1 to 4	Planner, Researcher, Writer, and Critic agents; prompt templates, benchmark runner.
Workstream 4	Deterministic Financial & Analytics Engine	Weeks 1 to 4	Mortgage amortizer, yield models, ROI projections, sensitivity scenario engine.
Workstream 5	Backend REST API & Microservices	Weeks 1 to 4	FastAPI service, `/api/v1/` endpoints, OpenAPI specifications, rate limiting.
Workstream 6	User Interface, Quality Assurance & DevOps	Weeks 1 to 4	Streamlit dashboard, Plotly visualizations, test suites, Docker Compose, PDF docs.

SECTION 6

Project's PDF Document Structure & Standards

Architectural layout, formatting standards, and visual governance guidelines for all project PDF deliverables across subsequent release iterations.

Publication Standard: All PDF reports generated for Atlas AQAR must conform to the unified publication layout detailed below. Subsequent iterations (v1.1, v1.2, and v2.0 Final) will maintain this exact structural framework while updating data metrics, deliverable statuses, and empirical proofs.

C **Cover Page** — *Page 1*
High-impact dark gradient presentation: project icon, English and Arabic titles, descriptive subtitle, metadata grid (Version, Program, Track, Date), pill tags, and confidentiality notice. Single full-bleed page.

T **Table of Contents (TOC)** — *Page 2*
Two-level hierarchical navigation listing all primary sections (§1 to §7) and second-level sub-sections (§2.1 to §5.5) with right-aligned section links and page identifiers.

1 **Section 1: Executive Summary** — *Page 3*
High-level project snapshot: core problem addressed, agentic solution concept, 3 key summary cards, empirical baseline achievements table, and document scope callout. Strictly budgeted to 1 page.

2 **Section 2: Project Overview** — *Pages 4–5*
Five comprehensive subsections: Problem Statement & Market Need (§2.1), Vision/Mission/Objectives chip cards (§2.2), Scope & Boundary Exclusions (§2.3), Technology Stack table (§2.4), and Measured Performance Baseline table (§2.5).

3 **Section 3: Stakeholder Analysis** — *Pages 6–7*
Complete stakeholder governance: Internal Stakeholders table (§3.1), External Stakeholders table (§3.2), 2×2 Power/Interest Matrix with 4 distinct quadrants (§3.3), Communication Plan table (§3.4), and Strategic Engagement Protocols (§3.5).

4 **Section 4: Project Plan** — *Pages 8–9*
End-to-end project management: 4-week Milestones timeline (§4.1), 12 Key Deliverables table with badge statuses (§4.2), 7-item Risk Register with severity coding and mitigations (§4.3), and Assumptions & Constraints cards (§4.4).

5 **Section 5: Work Breakdown Structure (WBS)** — *Pages 10–13*
Granular task allocation: 4 weekly sprint cards (§5.1–§5.4) broken down across 6 distinct functional workstreams with specific, measurable tasks, followed by the comprehensive Workstream Summary Matrix (§5.5).

6 **Section 6: Project's PDF Document Structure & Standards** — *Page 14*

The self-referential design specification detailing layout, typography, print-friendly styling rules, and guidelines for subsequent revisions and improvements.

7

Section 7: Appendix, Glossary & Revision Log

Pages 15–16

Reference materials: Glossary of technical terms (Agent, GraphState, RAG, ReAct, Evidence ID), Acronyms table, Revision History tracking log (v1.0 to v2.0), and official reference documentation links.

Document Typography & Styling Guidelines

TYPOGRAPHY & COLOR PALETTE

Body Font: Inter (9.5pt to 10pt print size, 1.6 line height).
Monospace Font: JetBrains Mono (code blocks, Evidence IDs).
Primary Navy: `#0E1B2E` (headings, headers, cover).
Accent Gold: `#C9973E` (borders, highlights, badges).
Neutral Surfaces: `#FFFFFF` on `#FAFBFC` page background.

PRINT & EXPORT SPECIFICATIONS

Page Dimensions: Standard A4 Portrait (210mm × 297mm).
Margins: 18mm top/bottom, 16mm left/right.
Page Break Rules: `page-break-before: always` on all major section roots; cards and tables use `page-break-inside: avoid`.
Color Fidelity: `-webkit-print-color-adjust: exact` enabled.

SECTION 7

Appendix, Glossary & Revision Log

Terminology definitions, abbreviation index, revision tracking log, and official references.

Glossary of Key Technical Terms

TERM	TECHNICAL DEFINITION WITHIN ATLAS AQAR
Agentic Workflow	An AI design pattern where an LLM coordinates reasoning, tool invocation, observation, and state updates iteratively rather than in a single-prompt response.
LangGraph	A specialized framework built on LangChain for building stateful, multi-actor applications with cyclic graph computation and persistence.
GraphState	The centralized, immutable Pydantic v2 data model passed between all agents in the workflow, tracking queries, sub-tasks, evidence records, drafts, and critique notes.
Evidence Arsenal	A structured repository of verified facts, listing specifications, and financial calculations collected during the research phase, each carrying a unique Evidence ID.
Evidence ID	A deterministic citation tag (e.g., `[E001]`, `[E002]`) assigned to raw data items and referenced inline throughout generated analytical reports.
Deterministic Financial Engine	A pure Python mathematical module executing mortgage amortization, ROI, and sensitivity analysis arithmetically, completely bypassing LLM probabilistic text generation.
ReAct Loop	A cognitive pattern (Reasoning + Acting) where the Researcher agent generates thoughts, executes tool actions, observes results, and loops until sufficient evidence is accumulated.
Critic Agent	An editorial inspection agent that evaluates draft reports against a 7-point rubric (factual grounding, math accuracy, completeness) and issues conditional approval or revision feedback.
ChromaDB	An open-source, embedded vector database used in Atlas AQAR to store and semantically retrieve high-dimensional embeddings of properties and market reports.
DEPI	Digital Egypt Pioneers Initiative — the national training and incubation initiative under which Atlas AQAR is developed within Round 5.

Acronyms & Abbreviations

ACRONYM	FULL EXPANSION	CONTEXT OF USAGE
AI / ML	Artificial Intelligence / Machine Learning	Core domain vertical
API	Application Programming Interface	FastAPI backend services
DTO	Data Transfer Object	Pydantic request/response schemas
LLM	Large Language Model	Reasoning model backbone
NHA	National Housing Authority	Institutional project partner
RAG	Retrieval-Augmented Generation	Grounded evidence retrieval
ROI	Return on Investment	Financial analytics metric
WBS	Work Breakdown Structure	Project task decomposition

Formal Revision Tracking Log

VERSION	RELEASE DATE	SCOPE OF CHANGES	REVISION DESCRIPTION
v1.0	October 2026	Complete Initial Baseline	Initial comprehensive release: Project Plan, Stakeholder Matrix, 4-Week 6-Workstream WBS, and PDF Publication Standards.
v1.1	Scheduled Week 2	§2.5, §4.2, §5.2	Incorporate Sprint 2 agent implementation outcomes, updated deliverable statuses (D04–D06), and interim benchmark results.
v1.2	Scheduled Week 3	§4.3, §5.3, §5.4	Update Risk Register with live integration findings; document FastAPI and Streamlit deployment specifications.
v2.0	Scheduled Week 4	Final Release (All)	Final production release: certified benchmark audit results, Docker container sign-off, user guide documentation, and delivery archive.

Official Project Reference Links

- **GitHub Repository:** https://github.com/nhahub/NHA-5-072
- **LangGraph Documentation:** https://langchain-ai.github.io/langgraph/
- **FastAPI Documentation:** https://fastapi.tiangolo.com/
- **ChromaDB Reference:** https://docs.trychroma.com/
- **Pydantic v2 Documentation:** https://docs.pydantic.dev/latest/
- **Streamlit Documentation:** https://docs.streamlit.io/

> **Next Scheduled Update (v1.1):** Scheduled at the conclusion of Sprint 2 (Week 2). It will update deliverable milestones D04, D05, and D06 with live code references and include empirical test logs from the agent

> evaluation suite.
